
TRANSKRIP REVIEW TEKNIS

Solar Tidal Station Berbasis XIAO ESP32-S3

Review rekomendasi konsultasi terhadap firmware aktual: A02YYUW, filtering, INA3221, Blynk.Edgent, penghematan energi, dan deep sleep.

Tanggal	5 Agustus 2026
Proyek	ESP31-S3 INA3221 / Stasiun Pasang Surut
Dokumen acuan	Transkrip_Konsultasi_Solar_Tidal_Station.pdf
Status	Review saja—tidak ada perubahan firmware atau build

PERMINTAAN PENGGUNA

Lakukan review terhadap transkrip konsultasi dan sesuaikan penilaiannya dengan kode proyek saat ini, tanpa langsung menerapkan perubahan atau menjalankan build.

Kesimpulan review

Rekomendasi di transkrip secara umum bagus dan relevan, tetapi firmware sekarang masih beroperasi sebagai monitor kontinu setiap satu detik. Arsitekturnya belum sesuai dengan target stasiun tenaga surya yang bangun setiap lima menit, mengukur, mengirim, lalu deep sleep.

Area	Rekomendasi transkrip	Firmware saat ini
Interval	Siklus 5 menit	Membaca dan mengirim setiap 1 detik
A02YYUW	50 sampel, median + MAD + trimmed mean	Menggunakan satu frame valid terakhir
Mode A02YYUW	RX ke GND, real-time	Dokumentasi proyek masih memakai RX floating/processed
Power sensor	Diputus load switch saat tidur	Sensor selalu mendapat daya
Temperatur	Direkam, koreksi opsional	Suhu/kelembapan direkam, belum mengoreksi jarak
INA3221	Single-shot lalu power-down	Continuous conversion <code>0x7127</code>
CH1 solar	Tegangan dan arus	Hanya tegangan
CH2 baterai	Tegangan dan signed net current	Hanya tegangan
CH3 sistem	Tegangan dan arus	Sudah membaca keduanya
Wi-Fi	Timeout lalu tidur	Blynk.Edgent mengelola koneksi secara terus-menerus
Penyimpanan gagal kirim	Backlog lokal	Belum tersedia
Deep sleep	Wajib untuk hemat energi	Belum tersedia

Bagian yang paling layak diterapkan

1. Filtering A02YYUW

Ini perubahan paling bernilai untuk kualitas pengukuran. Saat ini `readA02YYUW()` sudah benar dalam hal mencari header `0xFF`, memeriksa checksum, dan membatasi jarak ke 30–4500 mm. Namun, satu frame valid langsung menjadi hasil pengukuran. Untuk permukaan bergelombang, pantulan dinding, hujan, atau serangan, itu belum cukup kuat.

Adaptasi yang disarankan:

1. Nyalakan sensor.
2. Tunggu 500–1000 ms.
3. Buang 5–10 frame pertama.
4. Kumpulkan 50 frame valid.
5. Hitung median.
6. Hitung MAD.
7. Tolak sampel di luar batas berikut.

```
abs(sample - median) <= max(3 × 1.4826 × MAD, 20 mm)
```

8. Buang 10% nilai terendah dan tertinggi.
9. Hitung rata-rata sisanya.
10. Tandai hasil buruk jika sampel yang lolos kurang dari 60%.

Tidak disarankan menggunakan `std::vector` secara intensif pada siklus berulang. Array tetap seperti `uint16_t samples[50]` lebih cocok untuk mikrokontroler dan menghindari fragmentasi heap.

2. Mode real-time A02YYUW

Untuk filtering sendiri, rekomendasi RX A02YYUW ke GND masuk akal. Firmware sekarang tidak mengirim perintah apa pun ke sensor, sehingga pin TX ESP32 pada D6 sebenarnya tidak diperlukan untuk komunikasi.

Konfigurasi yang lebih bersih:

- A02YYUW TX → XIAO D7/RX.
- A02YYUW RX → GND.
- UART ESP32 dapat diinisialisasi hanya dengan RX, tanpa TX jika framework mengizinkan pin TX negatif.

Processed mode juga tetap bisa dipakai, tetapi kemudian 50 sampel sering berlebihan karena sensor sudah melakukan pemrosesan internal.

3. Load switch untuk A02YYUW

Sangat direkomendasikan jika deep sleep diterapkan. Membuat ESP32 tidur tetapi membiarkan A02YYUW terus menyala akan mengurangi manfaat penghematan secara signifikan.

D0 sekarang sudah bebas setelah fungsi LED dihapus, sehingga berpotensi dipakai sebagai `SENSOR_POWER_ENABLE`. Namun keputusan final harus mengikuti rangkaian load switch:

- Enable harus kompatibel dengan logika 3,3 V.
- Kondisi default saat boot harus OFF.
- Jangan menyalakan A02YYUW langsung dari GPIO.
- Pastikan tidak terjadi backfeeding melalui jalur UART.

```
power sensor ON
→ tunggu stabil
→ bersihkan buffer UART
→ buang frame awal
→ ambil dan filter sampel
→ power sensor OFF
```

4. Deep sleep lima menit

Ini perubahan arsitektur terbesar. Interval harus dihitung dari awal siklus:

```
sleep = 300 detik - durasi aktif
```

Bukan selalu tidur 300 detik setelah pekerjaan selesai. Data yang perlu bertahan di RTC memory antara siklus dapat mencakup nomor siklus, timestamp relatif, status backlog, dan jumlah kegagalan koneksi. Konfigurasi Wi-Fi serta token Blynk.Edgent sudah disimpan secara permanen oleh Edgent sehingga tidak perlu disimpan ulang oleh aplikasi.

Konflik penting dengan Blynk.Edgent

Blynk.Edgent dirancang sebagai state machine yang terus dijalankan melalui:

```
BlynkEdgent.run();
```

Deep sleep tetap dapat digunakan, tetapi perlu pendekatan khusus:

- Saat perangkat belum pernah diprovisioning, Edgent harus diberi waktu tanpa batas pendek agar pengguna dapat menyelesaikan provisioning.
- Setelah sudah diprovisioning, setiap bangun perangkat mencoba koneksi dalam batas waktu tertentu.
- Jika koneksi gagal, perangkat harus tidur kembali.
- Blynk.Air OTA hanya dapat diterima ketika perangkat sedang bangun dan online.

Jika perangkat hanya aktif sekitar 15–30 detik setiap lima menit, OTA masih mungkin, tetapi jendela penerimaannya sempit. Strategi yang lebih aman adalah menyediakan mode maintenance:

- Tombol BOOT ditekan ketika startup → perangkat tetap aktif untuk provisioning/OTA.
- Siklus normal → ukur, kirim, lalu tidur.
- Sesekali perangkat tetap online lebih lama untuk memeriksa OTA.

Konfigurasi Edgent bawaan yang sekarang dipakai mempunyai timeout jaringan/cloud sekitar 50 detik. Ini lebih lama daripada rekomendasi transkrip 10–15 detik dan perlu disesuaikan jika prioritas utama adalah penghematan baterai.

INA3221

Pemetaan channel sekarang sudah sama secara fungsi dasar:

- CH1: solar.
- CH2: baterai.
- CH3: sistem 5 V.

Namun firmware baru membaca CH1 tegangan solar, CH2 tegangan baterai, serta CH3 tegangan dan arus sistem. Jadi rekomendasi transkrip belum sepenuhnya terpenuhi. Untuk analisis energi lengkap perlu menambahkan:

- Register shunt CH1 `0x01`.
- Register shunt CH2 `0x03`.
- Resistor shunt masing-masing channel.
- Signed current untuk CH1, CH2, dan CH3.
- Perhitungan daya setiap channel.

Khusus CH2, arah arus harus dikalibrasi sehingga tanda positif/negatif jelas berarti charging atau discharging. Posisi shunt baterai juga menentukan apakah yang terbaca benar-benar arus bersih baterai.

Nilai resistor tidak sebaiknya memakai satu konstanta untuk semua channel tanpa pemeriksaan fisik:

```
constexpr float CH1_SHUNT_OHM = 0.1f;
constexpr float CH2_SHUNT_OHM = 0.1f;
constexpr float CH3_SHUNT_OHM = 0.1f;
```

Firmware sekarang menggunakan continuous conversion. Untuk siklus lima menit, single-shot lalu power-down lebih hemat. Konfigurasinya perlu mengikuti waktu konversi dan averaging agar firmware menunggu conversion-ready sebelum membaca register.

Kompensasi temperatur

SHT40 sudah tersedia dan dibaca, jadi fondasinya sudah ada. Koreksi temperatur belum perlu langsung dijadikan nilai utama. Untuk fase commissioning, kirim tiga nilai:

- Jarak hasil filtering tanpa koreksi.
- Temperatur udara.
- Jarak dengan koreksi temperatur sebagai pembanding.

Rumus rasio kecepatan suara masuk akal jika didahului kalibrasi pada temperatur referensi:

```
corrected =
    filtered * calibrationFactor *
    speedOfSound(currentTemperature) /
    speedOfSound(referenceTemperature);
```

SHT40 harus mengukur udara di dekat jalur ultrasonik, bukan udara panas di dalam enclosure. Jika penempatannya salah, koreksi matematis justru dapat memperburuk hasil. Kelembapan boleh direkam untuk diagnosis, tetapi pengaruh temperatur jauh lebih dominan.

Blynk dan kualitas data

Virtual pin saat ini V0–V8 sudah terpakai. Jangan langsung menambahkan semua parameter dalam transkrip sekaligus. Untuk tahap awal, parameter paling berguna adalah:

- Jumlah sampel valid.
- Jumlah sampel ditolak.
- MAD.
- Status kualitas.
- Jarak filtered raw.
- Durasi aktif.
- RSSI Wi-Fi.

`measurement_quality` sebaiknya direpresentasikan sebagai integer/enum agar mudah dipakai dalam event dan dashboard:

```
0 = invalid  
1 = poor  
2 = good
```

Penyimpanan lokal

Saran menyimpan backlog ketika internet gagal bagus, tetapi tidak perlu menjadi perubahan pertama. Implementasi yang aman membutuhkan format record tetap, timestamp yang valid, batas jumlah record, perlindungan terhadap penulisan flash berlebihan, dan penghapusan hanya setelah cloud mengonfirmasi pengiriman.

LittleFS cocok untuk backlog berukuran terbatas. Namun timestamp memerlukan sinkronisasi waktu; setelah deep sleep, waktu dapat dipertahankan lewat RTC atau disinkronkan ketika jaringan tersedia.

Catatan terhadap beberapa klaim transkrip

Beberapa angka konsumsi A02YYUW merupakan estimasi berdasarkan nilai maksimum/typical. Angka penghematan 95–97% masuk akal secara duty cycle, tetapi konsumsi sistem nyata tetap harus diukur karena regulator Waveshare, breakout INA3221, LED pada board, dan efisiensi konversi dapat mendominasi.

Pernyataan “ukur sebelum Wi-Fi agar noise tidak mengganggu” layak diterapkan. Namun pembacaan tegangan ketika Wi-Fi aktif juga berguna untuk mengetahui voltage sag. Jadi pengukuran terbaik adalah dua tahap:

1. INA3221 sebelum Wi-Fi.
2. INA3221 ketika Wi-Fi sedang aktif.

Urutan implementasi yang direkomendasikan

1. Filtering 50 sampel A02YYUW dan quality metrics.
2. Ubah A02YYUW ke real-time mode.
3. Tambahkan load switch sensor.
4. Lengkapi pembacaan arus CH1 dan CH2.
5. Ubah INA3221 menjadi single-shot.
6. Susun siklus bangun–ukur–kirim–tidur lima menit.
7. Tambahkan timeout koneksi yang kompatibel dengan Edgent.
8. Tambahkan mode maintenance untuk provisioning dan OTA.
9. Rekam jarak raw dan temperature-corrected secara paralel.
10. Tambahkan backlog lokal setelah siklus utama stabil.

Rekomendasi utama: jangan menerapkan seluruh transkrip dalam satu perubahan besar. Filtering, power switching, deep sleep, Edgent, dan backlog masing-masing mengubah state machine firmware. Implementasi bertahap akan jauh lebih mudah diuji dan dikalibrasi di lapangan.

Dokumen ini dibuat dari jawaban review teknis Codex pada percakapan proyek Solar Tidal Station. Tidak ada perubahan kode maupun build yang dilakukan sebagai bagian dari review tersebut.